```
=> {:status=>200, :body=>"OK"}
>> http_response.status
=> 200
>> http_response.body
=> "OK"
```

In fact, the hash syntax is just a shorthand for spelling out each allowed message individually:

```
>> allow(http_response).to receive(:status).and_return(200)
=> #<RSpec::Mocks::MessageExpectation #<Double "HTTPResponse">.status(any ↩
arguments)>
>> allow(http_response).to receive(:body).and_return('OK')
=> #<RSpec::Mocks::MessageExpectation #<Double "HTTPResponse">.body(any ↩
arguments)>
```

This more verbose syntax doesn't buy you much for simple stubs like these. But it will be vital in the next chapter, where we need more precision.

All of these stubs are simple. They watch for specific messages and return the same value each time they receive a given message. They don't act differently based on their arguments, and in fact they ignore their arguments:

```
>> http_response.status(:args, :are, :ignored)
=> 200
>> http_response.body(:blocks, :are, :also) { :ignored }
=> "OK"
```

In the next chapter, we'll talk about how to express expected parameters and return values more precisely.

Stubs like these help you test a specific kind of behavior—the kind that can be verified just by looking at return values. The method you're testing will typically do the following steps:

1. Query data from a dependency
2. Perform a computation on that data
3. Return a result

Your specs can check your object's behavior just by looking at the return value in step 3. All the stub has to do is hand back an appropriate reply to the query at step 1.

Sometimes, you need to test an object that doesn't fit into this pattern. In these situations, you can turn to another kind of test double designed for this use case: a mock object.